Project Outline

1. Split of Responsibilities b/w LMs and Humans

Stage	**LM**	**Tools / Checkers**	**Human**
Target Lean schema	N/A? Or we can use LMs to grab some examples/templates.	Enforce schema once defined.	(TODO) Define the required output form: state type, inputs, outputs, transition, output fn, theorem skeletons.
Initial Translation	Translate Verilog/spec into Lean.	Compile, parse errors, run structural and shallow semantic checks.	(TODO) Write prompts and freeze them before the main run.
Iterative Repair	Revise Lean based on checker feedback.	Return machine-readable diagnostics	Set iteration budget and prevent hidden manual editing during benchmark runs.
Reference correctness	Optional reference translation drafts.	Run test vectors, bounded checks, theorem compilation	Decide what counts as semantically Correct. Manually inspect ambiguous failures

Error Taxonomy	N/A	Surface recurring failure classes.	Label & Interpret failures

During the main benchmark, humans should not silently patch generated Lean files. Manual edits are allowed only in pilot debugging and in a separate manual correction effort measurement. Otherwise the results stop reflecting the LLM-plus-checker pipeline.

2. What should be LLM-driven vs human-driven

- **LLM**-driven: initial RTL-to-Lean translation; theorem skeleton generation; repair after compiler/checker feedback, alternative phrasing or normalization of generated Lean.
- **Checker**-driven: Lean compile/typecheck. missing declarations + undefined identifier + illegal placeholders + schema violations + simple width/interface sanity checks + optional lightweight behavioral checks on small modules.
- **Human**-driven: choosing modules and properties, designing the target Lean abstraction, deciding the allowed, prompt format, building the checker, creating ground-truth references for a subset, interpreting semantic failures; writing the final analysis.
- **Fallback** boundary: if direct translation fails too often, the human-designed AST/IR pipeline should take over the semantics-preserving part, while the LLM is restricted to softer tasks such as naming, theorem phrasing, or proof sketch generation.

3. Models

- Primary model: GPT-5.4. OpenAI's current model docs describe GPT-5.4 as the best frontier model for complex reasoning and coding, and their model guide says it is the most capable model in the GPT-5 family for these kinds of tasks.
- Secondary model: Gemini 2.5 Pro? Google's official Gemini API docs describe Gemini 2.5 Pro as their most advanced model for complex reasoning and coding,

which makes it a good cross-provider comparison. + This would be helpful to remove biases GPT might have during the self-refinement loop.

4. Core systems to evaluate

- Sys A: - one-shot direct translation: Verilog/spec to Lean in one pass.
- Sys B - translation plus **compiler**: use Lean/Lake only as the checker.
- Sys C - translation plus custom **checker**: add schema and shallow semantic checks beyond compilation.
    - Depending on your interest. Compiler itself might be more than sufficient
- Sys D - translation plus checker-guided repair: let the LLM revise for 1-3 rounds using diagnostics.
- Sys E - AST fallback: use a structured RTL parser/IR path for the semantics and compare it against the direct LLM path. (If we have time / if we actually need this)

5. Benchmark design

- Tier 1: Simple Comb modules:
    - Mux
    - Decoder/Encoder
    - Comparator
    - Small ALU Blocks
- Tier 2: Easy sequential modules:
    - Regs / counters
    - Shift registers
    - Accumulator
    - Tiny FSM?
- Tier 3: Moderately structure controllers
    - FIFO control
    - Arbiter
    - RegFile slice
    - Control Fragments
    - Multi-cycle ALUs

- Tier 4: Property-oriented Tasks
    - Thm. skeletons for mutex, RAW, ordering, ROB behaviors (in-order commit) etc.?
    - Sorting network
    - Crypto Core
    - Optimized adders/multipliers/dividers

Potential input conditions:

- Verilog only
- Verilog + short NL spec
- Verilog + required LEAN template or schema
- Verilog + checker feedback from a prev failed round (for refinement)

6. Quick Checker Design

Compiler success alone might be too weak as a file may compile while still violating the intended HW abstraction or silently omitting critical structure. Thus, we might implement a custom checker that can prove 1) Lean-side well-formedness and 2) Hardware-side faithfulness

- Level 0: Compiler & parser checks
    - Syntax errors, malformed declarations, undefined identifiers
- Level 1: Structural Checks
    - Required state/input/output declaration exist
    - Existence of transition and output functions
    - No thm. Placeholders (e.g. `sorry` keyword)
    - namespace/schema match
- Level 2: Shallow Semantic Checks
    - Width consistency
    - Reset case
    - Full coverage of FSM states

        - No missing next-state references
- Level 3: Lightweight Behavioral Checks
    - Only for tiny modules? Compare directly with sim traces.

7. Iterative Refinement Protocol

The loop should be machine-driven and (hopefully) reproducible.

- Round 0: LLM generates Lean directly from the prompt
- Check: Run lean compilation and the custom checker
- Normalize diagnostics into structured categories
    - This should be formalized at some point
- Round 1-3 (shot # can be varying): feed diagnostics back to the LLM and request a corrected lean file (Vericoder type shi?)
- Stop once the file passes all required checks or the iteration budget is exhausted

8. Metrics

Potential Candidates:

- Generation metrics
    - Parse success
    - Compilation success
    - Schema-conformance success
    - No-placeholder rate
- Semantic Metrics
    - Behavioral agreement on test vectors
    - Property pass rate
    - Theorem statement well-formedness
- Repair Metrics:
    - Success after zero-shot, few-shots, etc.
    - Average round to success

    - New-error introduction rate
- Human-effort metrics:
    - Mis or edits needed for manual correction no a sampled subset
- Errors
    - Frequencies of failures (syntax, typing, interface, width, operator, etc.)

We can also consider using LLM-as-a-judge method.

9. Implementation Plan

- Week 1: Freeze a common Lean target schema + Curate benchmark modules + write a few reference Lean models
- Week 2: Prompt Engineering + Checker implementation and diagnostics normalization
- Week 3: build the experiment harness (prompt -> generation -> compile/check -> repair loop system)
- Week 4: Run pilot experiments on 3-5 modules and refine prompts and checks
- Week 5 and onward: run the full bench + human error analysis